Course Outline: Building a Python API to Serve the Frontend

Title: Building a Python API to Serve the Frontend **Learnpack:** + Building a Python API to serve the frontend **Prerequisite:** Skill 18 — Designing Backend Architecture (FastAPI introduction, MVC, Python imports) **Target Audience:** Beginner developers who have just installed FastAPI and understand basic backend architecture **Total Lessons:** 10 **Estimated Total Length:** ~5,000 words **Format:** Conceptual (0% hands-on)

Course Description

In this course, students learn how to build a Python API that serves data to a frontend application. They already know what FastAPI is and how to install it from the previous skill. Now they put that knowledge into practice by understanding how to create endpoints that respond to HTTP requests, validate incoming data, and return clean responses.

Students will learn each CRUD operation individually (GET, POST, PUT, PATCH, DELETE) using in-memory data — no database yet. They will also learn why input validation and response serialization matter, and how Pydantic makes both nearly effortless. By the end, students will understand the full lifecycle of an API request.

This course deliberately avoids database integration, authentication, and API documentation — those are covered in upcoming skills.

Course Philosophy

This course emphasizes:

- **Path of least resistance:** Start with the simplest endpoint, then layer on complexity one verb at a time.
- **Frontend-first thinking:** Every endpoint exists because the frontend needs it.
- **Iteration over perfection:** Get a basic endpoint working first, then improve it.

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - CRUD Endpoints	6	~3,000
02 - Validation and Serialization	1	~550
03 - Assessment	1	~700
04 - Wrapping Up	1	~450
Total	**10**	**~5,150**

Section 00: Welcome

00.0 Welcome 🕮 (~450 words)

Learning Objectives:

- Identify what this course covers and what it does not (no database, no auth, no docs)
- Recognize where this course fits in the learning path (after architecture, before storage and auth)

Content Outline:

- What you will build: a working API that a frontend can consume
- The mental model: the frontend asks questions, the backend answers them
- What is deliberately excluded and why (database comes in Skill 22, auth in Skill 23, docs in the next learnpack on this same day)
- How this course connects to what you already know: HTTP verbs from Week 0, FastAPI basics from Skill 18, and API concepts from Skill 14

Transitions: This lesson sets the stage. Next, we start with the foundational concepts — what an API endpoint actually is and how CRUD maps to the HTTP verbs you already know.

Key Principles:

- An API is a contract between the frontend and the backend
- Every endpoint exists to serve a specific frontend need

Examples:

- A todo app frontend needs to list todos, create a new one, mark one as complete, and delete one — that is four endpoints
- A contact form on a website sends a POST request to your API — the API processes the data and responds

---

Section 01: CRUD Endpoints

01.0 APIs, Endpoints, and CRUD 🕮 (~500 words)

Learning Objectives:

- Define what an API endpoint is and explain its purpose
- Map each CRUD operation to its corresponding HTTP method
- Describe the structure of a FastAPI endpoint (decorator, function, return value)

Content Outline:

- Quick refresher: what is an API? (Students covered this in Skill 14 — keep it brief, 2-3 sentences max)
- What is an endpoint? A specific URL + HTTP method combination that does one thing
- CRUD and HTTP methods: GET = Read, POST = Create, PUT = Full update, PATCH = Partial update, DELETE = Remove
- Anatomy of a FastAPI endpoint: the decorator (`@app.get("/items")`), the function, and the return value

- How FastAPI automatically converts Python dictionaries to JSON responses
- Resources as nouns: `POST /users`, `GET /users` — the URL defines what, the method defines the action
- Anti-pattern: using verbs in URLs like `GET /getUsers` or `POST /createNewUser` — the HTTP method already carries the action, so the URL should only name the resource
- Anti-pattern: reinventing the wheel — do not try to parse JSON manually or build routing from scratch; FastAPI handles all of this for you

Transitions: Now that you understand the big picture, we will build each CRUD operation one at a time, starting with GET.

Key Principles:

- One endpoint, one responsibility
- The URL names the resource (noun), the HTTP method names the action (verb)
- Use what the framework gives you — FastAPI handles routing, serialization, and error responses

Examples:

- `GET /todos` returns the list of all todos — this is the "Read" in CRUD
- `POST /todos` receives a new todo in the request body and adds it — this is the "Create"
- Anti-pattern: `@app.get("/getContactById")` → Correct: `@app.get("/contacts/{contact_id}")`

01.1 GET Endpoints 🕮 (~500 words)

Learning Objectives:

- Build a GET endpoint that returns a list of resources
- Build a GET endpoint that returns a single resource by ID
- Return a 404 response when a resource is not found

Content Outline:

- The simplest endpoint: `@app.get("/contacts")` that returns an in-memory list
- In-memory data: we are storing everything in a Python list — data disappears when the server restarts, and that is fine for now (Skill 22 introduces persistence)
- Returning a single resource: `@app.get("/contacts/{contact_id}")` with a path parameter
- What happens when the resource does not exist: use `HTTPException(status_code=404)` to tell the frontend
- Anti-pattern: returning status 200 with `{"error": "not found"}` — the frontend relies on status codes to decide what to show the user; returning 200 for errors forces the frontend to parse the body just to detect failures
- The pattern: every endpoint follows a similar flow — receive request, find data, return response with the correct status code

Transitions: You can now read data from your API. Next, you will learn how to receive new data from the frontend with POST endpoints.

Key Principles:

- GET endpoints are read-only — they never modify data
- Always use proper HTTP status codes — 200 for success, 404 for not found
- In-memory data is a legitimate tool for prototyping and learning

Examples:

- `GET /contacts` returns `[{"id": 1, "name": "Alice"}, {"id": 2, "name": "Bob"}]`
- `GET /contacts/1` returns `{"id": 1, "name": "Alice"}`
- `GET /contacts/999` returns status 404 with `{"detail": "Contact not found"}`

01.2 POST Endpoints 🕮 (~500 words)

Learning Objectives:

- Build a POST endpoint that receives data in the request body
- Assign an ID to a new resource and add it to the in-memory list
- Return the newly created resource with the appropriate status code

Content Outline:

- What POST does: the frontend sends data in the request body, and the backend creates a new resource
- Receiving the body: FastAPI can accept a Pydantic model or raw parameters — for now, use a simple Pydantic model to define what fields are expected (this is a preview; validation is covered in depth in Section 02)
- Assigning an ID: since there is no database, generate an ID from the existing list
- Returning the new resource: return the complete object so the frontend can update its state immediately
- Status code 201: use `status_code=201` in the decorator to signal that a resource was created, not just that the request succeeded

Transitions: You can now create resources. But what happens when the frontend needs to update an existing one? That is where PUT comes in.

Key Principles:

- POST creates a new resource — the server assigns the ID, not the frontend
- Always return the created resource so the frontend does not need a second request
- Use status code 201 for creation, not 200

Examples:

- `POST /contacts` with body `{"name": "Charlie", "email": "charlie@example.com"}` creates a new contact and returns `{"id": 3, "name": "Charlie", "email": "charlie@example.com"}`
- The decorator: `@app.post("/contacts", status_code=201)`

01.3 PUT Endpoints 🕮 (~500 words)

Learning Objectives:

- Build a PUT endpoint that replaces all fields of an existing resource

- Explain what PUT means semantically (full replacement)
- Return 404 when the resource to update does not exist

Content Outline:

- What PUT does: the frontend sends a complete new version of the resource, and the backend replaces the old one entirely
- The key idea: PUT means "replace everything" — if the frontend sends `{"name": "Alice Updated"}` without an email field, the email should be gone (or the model should require it)
- Finding the resource: search by ID in the in-memory list, return 404 if not found
- Returning the updated resource: send back the full updated object so the frontend stays in sync
- When to use PUT: when the frontend has the complete object and wants to overwrite it

Transitions: PUT replaces everything. But sometimes the frontend only wants to change one field. That is what PATCH is for.

Key Principles:

- PUT = full replacement — every field is overwritten
- Always return the updated resource
- Return 404 if the resource does not exist — do not silently fail

Examples:

- `PUT /contacts/1` with body `{"name": "Alice Updated", "email": "alice.new@example.com"}` replaces all fields for contact 1
- `PUT /contacts/999` returns 404 because that contact does not exist

01.4 PATCH Endpoints 🕮 (~500 words)

Learning Objectives:

- Build a PATCH endpoint that updates only specific fields of a resource
- Explain the difference between PUT (full replace) and PATCH (partial update)
- Use optional fields in Pydantic to accept partial data

Content Outline:

- What PATCH does: the frontend sends only the fields it wants to change, and the backend updates just those
- The difference from PUT: PUT replaces the entire resource; PATCH modifies only what is sent
- Using optional fields: define the Pydantic model with `Optional` so the frontend can omit fields it does not want to change
- Applying the update: check which fields were actually sent (not `None`) and update only those
- When to use PATCH vs PUT: PATCH when the frontend has a form that edits one field; PUT when it has the complete object

Transitions: You can now create, read, and update resources. The last CRUD operation is DELETE.

Key Principles:

- PATCH = partial update — only the sent fields change, the rest stay as they are
- Use `Optional` fields in the model to make partial data valid
- Choose PUT or PATCH based on what the frontend is actually sending

Examples:

- `PATCH /contacts/1` with body `{"email": "newemail@example.com"}` updates only the email; the name stays the same
- Pydantic model: `class ContactPatch(BaseModel): name: Optional[str] = None; email: Optional[str] = None`

01.5 DELETE Endpoints 🕮 (~500 words)

Learning Objectives:

- Build a DELETE endpoint that removes a resource by ID
- Return appropriate confirmation and status codes
- Return 404 when the resource to delete does not exist

Content Outline:

- What DELETE does: the frontend sends a request with the resource ID, and the backend removes it
- Finding and removing: search by ID in the in-memory list, remove it, return confirmation
- What to return: a simple message like `{"detail": "Contact deleted"}` — some APIs return the deleted object, others return just a confirmation; pick a convention and be consistent
- Return 404 if the resource does not exist — do not pretend the deletion succeeded
- Anti-pattern: returning 200 with `{"error": "not found"}` instead of a proper 404 — this forces the frontend to parse the response body to detect failures

Transitions: You now have all five CRUD operations. Your API can read, create, update, and delete resources. But right now it blindly trusts whatever data the frontend sends. In the next section, you will learn how to validate input and control what goes back out.

Key Principles:

- DELETE removes a resource — return confirmation, not the full object (unless your team agrees otherwise)
- Return 404 if the resource does not exist
- Be consistent: pick a response convention and stick with it across all endpoints

Examples:

- `DELETE /contacts/1` returns `{"detail": "Contact deleted"}`
- `DELETE /contacts/999` returns status 404 with `{"detail": "Contact not found"}`

Section 02: Validation and Serialization

02.0 Pydantic, Response Models, and Common Mistakes 🕮 (~550 words)

Learning Objectives:

- Explain why backend validation is necessary even when the frontend validates too
- Use Pydantic models to validate incoming data and serialize outgoing responses
- Identify common mistakes when building APIs (missing validation, exposing internal fields)

Content Outline:

- Why validate on the backend? The frontend is not the only client — APIs can be called from Postman, scripts, mobile apps, or malicious actors. Never trust incoming data.
- What Pydantic does: you define a model with type annotations, FastAPI uses it to automatically reject invalid requests with a 422 error
- Anti-pattern: not validating input — without a Pydantic model, someone sends `{"name": null}` and your API stores garbage data or crashes
- Serialization: converting your internal data into the format the frontend expects
- Why not return everything? Some fields are internal (timestamps, hashed passwords, internal flags) — the frontend should never see them
- Using `response_model` in FastAPI: declare which fields go out, and FastAPI strips the rest automatically
- Anti-pattern: exposing internal fields — returning `hashed_password` or `internal_notes` because you returned the raw object instead of using a response model
- Consistency: if `GET /contacts` returns `{id, name, email}`, every contact in every endpoint should have the same shape — no surprises for the frontend

Transitions: You now understand the complete lifecycle of an API request: receive it, validate the input, perform the action, and serialize the response. Time to test your knowledge.

Key Principles:

- Never trust the frontend — always validate on the backend
- `response_model` is your contract with the frontend — only expose what it needs
- Consistency in response shapes prevents frontend bugs

Examples:

- A `ContactCreate` model with `name: str` and `email: str` — if someone sends `{"name": 123}`, FastAPI rejects it with 422
- A `User` object has `id`, `name`, `email`, `hashed_password` — the `UserResponse` model only exposes `id`, `name`, `email`
- Anti-pattern: one endpoint returns `{"user_name": "Alice"}` and another returns `{"name": "Alice"}` — inconsistent naming breaks the frontend

---

Section 03: Assessment

03.0 Knowledge Check 🧐 (~700 words)

Learning Objectives:

- Demonstrate understanding of CRUD operations and their HTTP method mappings
- Identify correct validation and serialization practices

- Recognize common API anti-patterns

Question Format: Scenario-based (all questions)

Questions:

Question 1: A frontend developer tells you: "I need to get a list of all products." Which endpoint should you build?

A) `POST /products` B) `GET /getProducts` C) `GET /products` D) `GET /product/all`

Correct Answer: C — `GET /products`. The URL should be a plural noun (the resource), and GET is the method for reading. Option B uses a verb in the URL, which breaks REST conventions.

Question 2: You build a `POST /users` endpoint but do not add any Pydantic model for validation. A frontend sends `{"name": null, "email": 12345}`. What happens?

A) FastAPI automatically rejects the request with a 422 error B) The endpoint creates a user with `name=null` and `email=12345` C) The server crashes with a 500 error D) FastAPI converts the values to the correct types automatically

Correct Answer: B — Without a Pydantic model, FastAPI does not validate the body. Whatever the frontend sends goes through, even if the data makes no sense.

Question 3: Your `DELETE /contacts/999` endpoint cannot find contact 999. What should it return?

A) Status 200 with `{"error": "not found"}` B) Status 404 with `{"detail": "Contact not found"}` C) Status 500 with `{"detail": "Server error"}` D) Status 204 with no body

Correct Answer: B — A 404 tells the frontend the resource does not exist. Returning 200 with an error message forces the frontend to parse the body to detect failures.

Question 4: You have a `User` object with fields: `id`, `name`, `email`, `hashed_password`, `internal_role`. The frontend only needs `id`, `name`, and `email`. What should you do?

A) Return the full object and let the frontend ignore the extra fields B) Manually delete fields from the dictionary before returning C) Use a Pydantic `response_model` that only includes `id`, `name`, and `email` D) Create a separate endpoint that returns fewer fields

Correct Answer: C — A `response_model` ensures sensitive data is never accidentally exposed. FastAPI strips extra fields automatically.

Question 5: What is the difference between PUT and PATCH?

A) PUT creates a resource, PATCH deletes it B) PUT replaces the entire resource, PATCH updates only the fields that were sent C) PUT is for public endpoints, PATCH is for private ones D) There is no difference, they are interchangeable

Correct Answer: B — PUT means full replacement (every field is overwritten), PATCH means partial update (only the sent fields change).

Question 6: You are building an API with in-memory data (a Python list). What is the main limitation?

A) In-memory data cannot store more than 100 items B) In-memory data is lost when the server restarts C) In-memory data cannot be accessed by GET endpoints D) In-memory data is slower than a database

Correct Answer: B — Data in a Python list exists only in RAM. When the server restarts, everything is gone. Persistence is covered in Skill 22.

Evaluation Criteria:

- 6/6: Excellent — solid understanding of API fundamentals
- 4-5/6: Good — review the areas you missed
- 2-3/6: Needs review — revisit the lessons on the topics you struggled with
- 0-1/6: Start over — re-read the course material before moving on

---

Section 04: Wrapping Up

04.0 What's Next (~450 words)

Content Outline:

- Course recap: you learned what endpoints are, built each CRUD operation individually (GET, POST, PUT, PATCH, DELETE), validated input with Pydantic, and controlled output with response models
- What you can do now: design and reason about a complete API for any resource, understand the request lifecycle, and identify common mistakes before they happen
- Connection to the bigger picture: right now your API forgets everything when it restarts — Skill 22 introduces persistence with TinyDB, Skill 23 adds authentication, and the next learnpack today covers API documentation
- Resources: FastAPI official documentation (fastapi.tiangolo.com), Pydantic docs, HTTP status code reference
- Encouragement: you now understand the backbone of every web application — the API layer that connects the frontend to the backend. Every skill from here builds on this foundation.

Note: This is conclusion only — no theory, exercises, or assessment.